浅谈 PageRank 算法

唐怡　姚鑫婷　李知夏

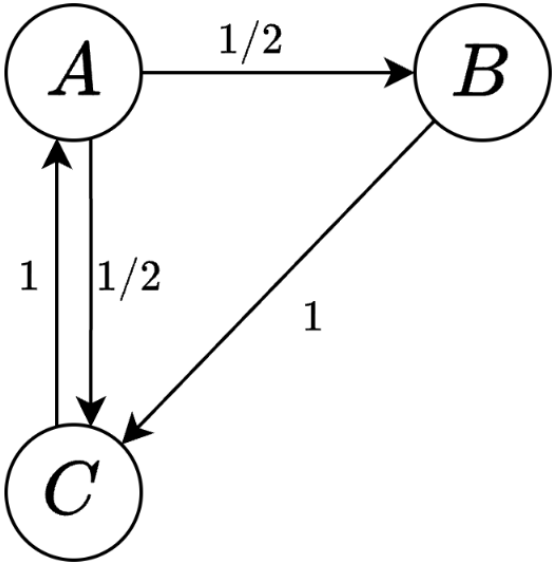
湖南大学 计算机学院

2026 年 3 月 19 日

1.1 图上随机游走

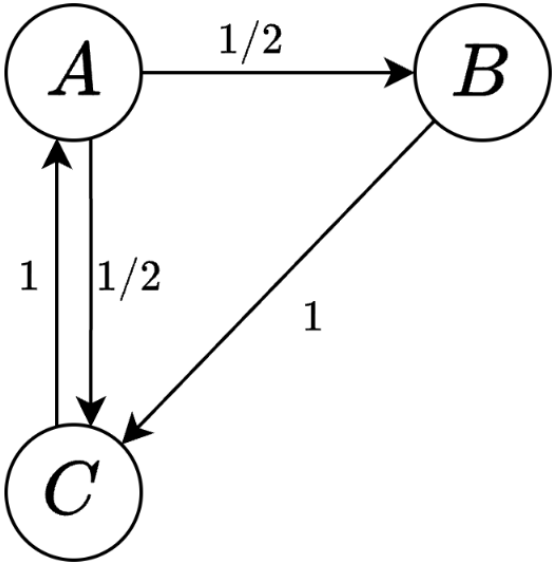
PageRank 将网页之间的引用关系视为一个有向图，这是一个在图上跑的算法。

1.1 图上随机游走



PageRank 认为浏览者是在网页上进行随机游走的，对应到有向图上，就是在有向图上随机游走。

1.1 图上随机游走



PageRank 认为浏览者是在网页上进行随机游走的，对应到有向图上，就是在有向图上随机游走。由于是随机游走，所以从一个节点出发，到达相邻节点的概率是相同的。

1.1 图上随机游走

记 $p_{i,j}$ 为第 i 轮后，位于节点 j 的概率。

初始时，位于每个节点的概率相同，即 $p_{0,i} = \frac{1}{n}$。

1.1 图上随机游走

记 $p_{i,j}$ 为第 i 轮后，位于节点 j 的概率。

初始时，位于每个节点的概率相同，即 $p_{0,i} = \frac{1}{n}$。

从第 i 轮到第 $i+1$ 轮，有状态转移：

$$p_{i+1,v} = \sum_{u} p_{i,u} \cdot T_{u,v}$$

1.1 图上随机游走

该过程可以用矩阵表示。比如上图的例子，转为矩阵表示：

$$\mathbf{G} = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

1.1 图上随机游走

该过程可以用矩阵表示。比如上图的例子，转为矩阵表示：

$$\mathbf{G} = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

按行归一化，则得到从一个点出去到别的点的概率：

$$\mathbf{T} = \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

1.1 图上随机游走

初始时我们有行向量，表示位于各个节点的概率：

$$\mathbf{V} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix}$$

1.1 图上随机游走

初始时我们有行向量，表示位于各个节点的概率：

$$\mathbf{V} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix}$$

行向量乘以矩阵会得到一个新的行向量。矩阵乘法的过程和上述状态转移含义相同：

$$\mathbf{VT} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix} \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{3} & \frac{1}{6} & \frac{1}{2} \end{bmatrix} = \mathbf{V}'$$

1.1 图上随机游走

于是，我们不断地右乘矩阵 $\mathbf{T}$，即可得到一直随机游走后，到达每个节点的概率。

1.1 图上随机游走

于是，我们不断地右乘矩阵 $\mathbf{T}$，即可得到一直随机游走后，到达每个节点的概率。

我们将每个节点的概率从大到小排，就相当于完成了对节点的排名。

1.2 等级泄露

等级泄露指的是，如果有节点出度为 0，则意味着概率会不断地累积到这个节点。那么随着这个随机游走的过程不断进行下去，这个节点会不断地”吸干“别的节点的权重。

1.2 等级泄露

等级泄露指的是，如果有节点出度为 0，则意味着概率会不断地累积到这个节点。那么随着这个随机游走的过程不断进行下去，这个节点会不断地”吸干“别的节点的权重。

我们将出度为 0 的节点称为 dangling node，其组成集合为 danglingNodes。

1.2 等级泄露

等级泄露指的是，如果有节点出度为 0，则意味着概率会不断地累积到这个节点。那么随着这个随机游走的过程不断进行下去，这个节点会不断地”吸干“别的节点的权重。

我们将出度为 0 的节点称为 dangling node，其组成集合为 danglingNodes。

走不出去的概率：

$$leakRank = \sum_{i \in \text{danglingNodes}} \mathbf{V}_i$$

1.2 等级泄露

PageRank 算法考虑，如果一个人浏览到一个没有另外跳转链接的页面，那么这个人下一步可能会随机跳转到所有的界面上。

1.2 等级泄露

PageRank 算法考虑，如果一个人浏览到一个没有另外跳转链接的页面，那么这个人下一步可能会随机跳转到所有的界面上。

我们这种随机跳转时，每个节点的权重组成行向量 **danglingWeights**，那么就相当于，将 $leakRank$ 按权重 **danglingWeights** 重新分配。

1.2 等级泄露

PageRank 算法考虑，如果一个人浏览到一个没有另外跳转链接的页面，那么这个人下一步可能会随机跳转到所有的界面上。

我们这种随机跳转时，每个节点的权重组成行向量 **danglingWeights**，那么就相当于，将 $leakRank$ 按权重 **danglingWeights** 重新分配。

于是原本的 PageRank 算法为：

$$\mathbf{VT} + leakRank \cdot \mathbf{danglingWeights} = \mathbf{V}'$$

1.3 等级沉默

如果一个点入度为 0，则没有节点传给他概率，那么随着随机过程的进行，这个节点的权重会归零。

1.3 等级沉默

如果一个点入度为 0，则没有节点传给他概率，那么随着随机过程的进行，这个节点的权重会归零。

PageRank 算法提出了一个常数因子 $followingProb$（又称为阻尼系数，常见取值为 0.85），表示用户有多少概率会从当前节点上继续跳转到其他节点。

1.3 等级沉默

如果一个点入度为 0，则没有节点传给他概率，那么随着随机过程的进行，这个节点的权重会归零。

PageRank 算法提出了一个常数因子 $followingProb$（又称为阻尼系数，常见取值为 0.85），表示用户有多少概率会从当前节点上继续跳转到其他节点。

那么 $1 - followingProb$ 就表示用户有多少概率会跳转到一个随机的节点。

1.3 等级沉默

对于后者，随机跳转时每个节点也会有一个权重，每个节点的权重组成行向量 **personalization**。

1.3 等级沉默

对于后者，随机跳转时每个节点也会有一个权重，每个节点的权重组成行向量 **personalization**。

于是原本的 PageRank 算法为：

$$followingProb \cdot (\mathbf{VT} + leakRank \cdot \mathbf{danglingWeights}) + (1 - followingProb) \cdot \mathbf{personalization} = \mathbf{V}'$$

1.4 算法收敛

可以证明，PageRank 算法是可以收敛的。

1.4 算法收敛

$$followingProb \cdot (\mathbf{VT} + leakRank \cdot \mathbf{danglingWeights}) + (1 - followingProb) \cdot \mathbf{personalization} = \mathbf{V}'$$

1.4 算法收敛

$$followingProb \cdot (\mathbf{VT} + leakRank \cdot \mathbf{danglingWeights}) + (1 - followingProb) \cdot \mathbf{personalization} = \mathbf{V}'$$

如果我们要证明上式收敛，那么只需要证明 **VT** 在这个算法过程中收敛。

1.4 算法收敛

$$followingProb \cdot (\mathbf{VT} + leakRank \cdot \mathbf{danglingWeights}) + (1 - followingProb) \cdot \mathbf{personalization} = \mathbf{V}'$$

如果我们要证明上式收敛，那么只需要证明 **VT** 在这个算法过程中收敛。

$(1 - followingProb) \cdot$ **personalization** 是常数

1.4 算法收敛

$$followingProb \cdot (\mathbf{VT} + leakRank \cdot \mathbf{danglingWeights}) + (1 - followingProb) \cdot \mathbf{personalization} = \mathbf{V}'$$

如果我们要证明上式收敛，那么只需要证明 **VT** 在这个算法过程中收敛。

$(1 - followingProb) \cdot$ **personalization** 是常数

对于 $leakRank \cdot$ **danglingWeights** 部分，我们可以不严谨地认为，假如说 **VT** 收敛了，那么 $leakRank \cdot$ **danglingWeights** 部分也就是常数了。

1.4 算法收敛

VT 可以看成是一个马尔科夫链过程

1.4 算法收敛

$\mathbf{VT}$ 可以看成是一个马尔科夫链过程

$\mathbf{V}_\infty = \lim_{n\to\infty} \mathbf{V}_0\mathbf{T}^n$。$\mathbf{V}_0$ 为初始时各个节点的 PageRank 值。

1.4 算法收敛

$\mathbf{VT}$ 可以看成是一个马尔科夫链过程

$\mathbf{V}_\infty = \lim_{n\to\infty} \mathbf{V}_0\mathbf{T}^n$。$\mathbf{V}_0$ 为初始时各个节点的 PageRank 值。

现在证明 $\lim_{n\to\infty} \mathbf{T}^n$ 是收敛的。

1.4 算法收敛

考虑将矩阵 $\mathbf{T}$ 做特征值分解:

$$\mathbf{T} = \mathbf{X}\Lambda\mathbf{X}^{-1}$$

1.4 算法收敛

考虑将矩阵 $\mathbf{T}$ 做特征值分解:

$$\mathbf{T} = \mathbf{X}\Lambda\mathbf{X}^{-1}$$

然后有:

$$\mathbf{T}^n = \mathbf{X}\Lambda^n\mathbf{X}^{-1}$$

1.4 算法收敛

考虑将矩阵 $\mathbf{T}$ 做特征值分解：

$$\mathbf{T} = \mathbf{X}\Lambda\mathbf{X}^{-1}$$

然后有：

$$\mathbf{T}^n = \mathbf{X}\Lambda^n\mathbf{X}^{-1}$$

于是考虑 Λ^n 的收敛性。Λ 是对角矩阵。主对角线上的元素是由 $\mathbf{T}$ 的特征值组成的。

1.4 算法收敛

考虑将矩阵 $\mathbf{T}$ 做特征值分解：

$$\mathbf{T} = \mathbf{X}\Lambda\mathbf{X}^{-1}$$

然后有：

$$\mathbf{T}^n = \mathbf{X}\Lambda^n\mathbf{X}^{-1}$$

于是考虑 Λ^n 的收敛性。Λ 是对角矩阵。主对角线上的元素是由 $\mathbf{T}$ 的特征值组成的。

事实上，$\mathbf{T}$ 的特征值 λ_i 必然满足 $|\lambda_i| \leq 1$，于是 Λ^n 是收敛的。

1.4 算法收敛

下面证明 $\mathbf{T}$ 的特征值 λ_i 必然满足 $|\lambda_i| \leq 1$:

1.4 算法收敛

下面证明 $\mathbf{T}$ 的特征值 λ_i 必然满足 $|\lambda_i| \leq 1$:

我们称 $\mathbf{T}$ 为马尔可夫矩阵，当且仅当它每一行的元素之和为 1

1.4 算法收敛

下面证明 $\mathbf{T}$ 的特征值 λ_i 必然满足 $|\lambda_i| \leq 1$:

我们称 $\mathbf{T}$ 为马尔可夫矩阵，当且仅当它每一行的元素之和为 1

- 我们这里是右乘马尔可夫矩阵，所以要求每一行之和为 1

1.4 算法收敛

下面证明 $\mathbf{T}$ 的特征值 λ_i 必然满足 $|\lambda_i| \leq 1$:

我们称 $\mathbf{T}$ 为马尔可夫矩阵，当且仅当它每一行的元素之和为 1

- 我们这里是右乘马尔可夫矩阵，所以要求每一行之和为 1
- 矩阵中每一项都是概率，$0 \leq \mathbf{T}_{ij} \leq 1$

1.4 算法收敛

下面证明 $\mathbf{T}$ 的特征值 λ_i 必然满足 $|\lambda_i| \leq 1$:

我们称 $\mathbf{T}$ 为马尔可夫矩阵，当且仅当它每一行的元素之和为 1

- 我们这里是右乘马尔可夫矩阵，所以要求每一行之和为 1
- 矩阵中每一项都是概率，$0 \leq \mathbf{T}_{ij} \leq 1$
- 简单起见，这里不考虑 dangling nodes。

1.4 算法收敛

下面证明 $\mathbf{T}$ 的特征值 λ_i 必然满足 $|\lambda_i| \leq 1$：

我们称 $\mathbf{T}$ 为马尔可夫矩阵，当且仅当它每一行的元素之和为 1

- 我们这里是右乘马尔可夫矩阵，所以要求每一行之和为 1
- 矩阵中每一项都是概率，$0 \leq \mathbf{T}_{ij} \leq 1$
- 简单起见，这里不考虑 dangling nodes。

易知：两个马尔可夫矩阵相乘，结果也是马尔可夫矩阵。

1.4 算法收敛

令 $\mathbf{T}$ 中绝对值最大的特征值为 λ_1，其对应的特征向量为 $\mathbf{x}$，根据特征值定义，有：

$$\mathbf{T}\mathbf{x} = \lambda_1 \mathbf{x}$$

1.4 算法收敛

令 $\mathbf{T}$ 中绝对值最大的特征值为 λ_1，其对应的特征向量为 $\mathbf{x}$，根据特征值定义，有：

$$\mathbf{T}\mathbf{x} = \lambda_1 \mathbf{x}$$

于是有：

$$\begin{aligned}
\mathbf{T}^n\mathbf{x} &= \mathbf{T}^{n-1} \cdot \mathbf{T} \cdot \mathbf{x} \\
&= \mathbf{T}^{n-1} \cdot \lambda_1 \mathbf{x} \\
&= \lambda_1 \mathbf{T}^{n-1} \cdot \mathbf{x} \\
&\cdots \\
&= \lambda_1^n \mathbf{x}
\end{aligned}$$

1.4 算法收敛

假如说 $|\lambda_1| > 1$，那么 $\lim_{n\to\infty} \lambda_1^n \mathbf{x} = \infty$。

1.4 算法收敛

假如说 $|\lambda_1| > 1$，那么 $\lim_{n\to\infty} \lambda_1^n \mathbf{x} = \infty$。

而 $\mathbf{T}^n$ 的每个元素都是位于 0 和 1 之间的，所以显然趋近不了无穷，因此假设不成立。

1.4 算法收敛

假如说 $|\lambda_1| > 1$，那么 $\lim_{n\to\infty} \lambda_1^n \mathbf{x} = \infty$。

而 $\mathbf{T}^n$ 的每个元素都是位于 0 和 1 之间的，所以显然趋近不了无穷，因此假设不成立。

所以 $\mathbf{T}$ 的特征值 λ_i 必然满足 $|\lambda_i| \leq 1$。

1.4 算法收敛

如何判定收敛?

1.4 算法收敛

如何判定收敛?

Talk is cheap, show me your code.

1.5 时间复杂度

令矩阵运算的时间复杂度为 M，迭代次数为 d，那么时间复杂度就为 dM。

1.5 时间复杂度

令矩阵运算的时间复杂度为 M，迭代次数为 d，那么时间复杂度就为 dM。

M？视代码实现而定。

1.5 时间复杂度

令矩阵运算的时间复杂度为 M，迭代次数为 d，那么时间复杂度就为 dM。

M？视代码实现而定。

d?

1.5 时间复杂度

令矩阵运算的时间复杂度为 M，迭代次数为 d，那么时间复杂度就为 dM。

M？视代码实现而定。

d?

- 有人说是 $\log$ 级别的（通过实验得出的结果）。

1.5 时间复杂度

令矩阵运算的时间复杂度为 M，迭代次数为 d，那么时间复杂度就为 dM。

M？视代码实现而定。

d?

- 有人说是 $\log$ 级别的（通过实验得出的结果）。
- 我们在实际中发现，迭代次数是比较小的，一般来说可能个位数的迭代次数就差不多了。

1.5 时间复杂度

令矩阵运算的时间复杂度为 M，迭代次数为 d，那么时间复杂度就为 dM。

M？视代码实现而定。

d?

- 有人说是 $\log$ 级别的（通过实验得出的结果）。
- 我们在实际中发现，迭代次数是比较小的，一般来说可能个位数的迭代次数就差不多了。
- 我们随后又发现，迭代次数和 $followingProb$ 有关，$followingProb$ 比较大的话，迭代次数也会变得比较多。

2. 代码实现

GitHub：https://github.com/caiwen666/hnu-algo

2.1 朴素实现

我们使用 Rust。考虑到矩阵乘法的实现并不是该算法的重点，我们直接使用 Rust 的 ndarray crate 进行矩阵运算：

2.1 朴素实现

我们使用 Rust。考虑到矩阵乘法的实现并不是该算法的重点，我们直接使用 Rust 的 ndarray crate 进行矩阵运算：

不过建图时有一个细节需要注意，如果 u 向 v 有重边，那么 $\mathbf{G}_{u,v}$ 还是为 1。

2.1 朴素实现

```
pub fn rank(&self, following_prob: f64, tolerance: f64) -> Vec<(T, f64)>
      {
^^I...
    // 初始化 PageRank
    let mut last = Array2::<f64>::from_elem((1, self.capacity), 1.0 /
     self.capacity as f64);
    let personalization = last.clone();
    let dangling_weights = personalization.clone();
    // 不断迭代直到达到收敛条件
    loop {
        let t = last.select(Axis(1), dangling_nodes_index.as_slice());
        let leak_rank = t.sum();
        let current = following_prob
            * (last.dot(&transition_matrix) + leak_rank * &
     dangling_weights)
            + (1.0 - following_prob) * &personalization;
        // CALLBACK: 收敛判定
        if (&last - &current).abs().sum() < self.capacity as f64 *
     tolerance {
            break;
        }
        last = current;
    }
^^I
```

2.1 朴素实现

我们在三体人物关系的图上跑了一下 PageRank 算法。

2.1 朴素实现

我们在三体人物关系的图上跑了一下 PageRank 算法。

数据来源：https://textdata.cn/blog/2022-11-29-santi-relationship-visualization-with-pyecharts/

2.1 朴素实现

我们在三体人物关系的图上跑了一下 PageRank 算法。

数据来源：https://textdata.cn/blog/2022-11-29-santi-relationship-visualization-with-pyecharts/

测试代码在 `/tests/pagerank.rs` 中。

2.1 朴素实现

```
[("罗辑", 0.06317139467552539), ("叶文洁", 0.06088064117254309), ("程心",
     0.05211684856535334), ("汪淼", 0.04618440510334715), (" 云天明",
    0.0407062044364546), ("智子", 0.035534663924738), ("章北海",
    0.024534285949243787),
...
```

2.1 朴素实现

```
[("罗辑", 0.06317139467552539), ("叶文洁", 0.06088064117254309), ("程心",
     0.05211684856535334), ("汪淼", 0.04618440510334715), (" 云天明",
    0.0407062044364546), ("智子", 0.035534663924738), ("章北海",
    0.024534285949243787),
...
```

从直觉上来看是没问题的，众所周知罗辑是三体中的核心人物...

2.1 朴素实现

不过我们还有另一个思考：如果我们直接把图上节点按度数（原图是无向图）从大到小排序，貌似也能得到一个差不多的：

2.1 朴素实现

不过我们还有另一个思考：如果我们直接把图上节点按度数（原图是无向图）从大到小排序，貌似也能得到一个差不多的：

```
[('叶文洁', 34), ('罗辑', 34), ('程心', 28), ('汪淼', 26), ('云天明', 22)
    , ('智子', 20), ('史强', 12), ('章北海', 12),
...
```

2.1 朴素实现

不过我们还有另一个思考：如果我们直接把图上节点按度数（原图是无向图）从大到小排序，貌似也能得到一个差不多的：

```
[('叶文洁', 34), ('罗辑', 34), ('程心', 28), ('汪淼', 26), ('云天明', 22)
    , ('智子', 20), ('史强', 12), ('章北海', 12),
...
```

在这个数据集上，PageRank 的效果看起来并没有很显著。

2.1 朴素实现

不过我们还有另一个思考：如果我们直接把图上节点按度数（原图是无向图）从大到小排序，貌似也能得到一个差不多的：

```
[('叶文洁', 34), ('罗辑', 34), ('程心', 28), ('汪淼', 26), ('云天明', 22)
    , ('智子', 20), ('史强', 12), ('章北海', 12),
...
```

在这个数据集上，PageRank 的效果看起来并没有很显著。

直观地理解，PageRank 通常用来缓解这种情况：如果一个节点被很多节点引用，但这些引用它的节点都不重要，那么即使它被引用得很多，重要性也不高。而对于三体人物，重要的人物能接触到的人自然更多，不重要的人物能接触到的人也更少。

2.1 朴素实现

我们还在一个神秘的数据集上跑了一下 PageRank：

2.1 朴素实现

我们还在一个神秘的数据集上跑了一下 PageRank：

数据来源：https://snap.stanford.edu/data/web-Google.html

2.1 朴素实现

我们还在一个神秘的数据集上跑了一下 PageRank：

数据来源：https://snap.stanford.edu/data/web-Google.html

网站上说这是 Google 搜索引擎爬取到的真实的网站引用关系的数据，但他把网站链接映射成编号了，趣味性有点低...

2.1 朴素实现

然后不出意外 OOM 了，这个数据集有 875713 个点，我们肯定是存不下 875713×875713 大小的二维数组的。

2.1 朴素实现

CALLBACK：行向量乘矩阵的时间复杂度为 $O(n^2)$。

2.2 稀疏矩阵

注意到，该数据集上只有 5105039 条边，远小于 875713×875713，这个图很稀疏。

2.2 稀疏矩阵

注意到，该数据集上只有 5105039 条边，远小于 875713×875713，这个图很稀疏。

稀疏矩阵乘法的技巧详情可以参考：
https://www.caiwen.work/post/mit6172-1#1-5-sparsity

2.2 稀疏矩阵

注意到，该数据集上只有 5105039 条边，远小于 875713×875713，这个图很稀疏。

稀疏矩阵乘法的技巧详情可以参考：
https://www.caiwen.work/post/mit6172-1#1-5-sparsity

由于我们是行向量乘矩阵，这里我们把列压缩掉。

2.2 稀疏矩阵

```
// algorithms/matrix.rs
pub struct CSCMatrix<T> {
    col_elements: Vec<HashMap<usize, T>>,
    row: usize,
    col: usize,
}

pub fn left_mul(&self, vector: Vec<T>) -> Vec<T> {
    if vector.len() != self.row {
        panic!("Vector length must be equal to matrix row");
    }
    let mut result = vec![T::default(); self.col];
    for (col, col_list) in self.col_elements.iter().enumerate() {
        for (row, value) in col_list.iter() {
            result[col] = result[col] + vector[*row] * *value;
        }
    }
    result
}
```

2.2 稀疏矩阵

我们只对非 0 元素进行存储并运算。非 0 元素的数量大概是边数 m，于是：

- 空间复杂度：$O(n^2) \to O(m)$
- 时间复杂度：$O(n^2) \to O(nm)$

2.2 稀疏矩阵

然后再跑一下上述的 Google 的数据集，我们得到：

节点	PageRank 值	入度度数排名
163075	0.00095211233337705 86	7
597621	0.0009013686628242723	2
537039	0.000895381572659312	1
837478	0.0008761661604315241	10+
885605	0.0008216087428297469	6

2.2 稀疏矩阵

然后再跑一下上述的 Google 的数据集，我们得到：

节点	PageRank 值	入度度数排名
163075	0.0009521123333770586	7
597621	0.0009013686628242723	2
537039	0.000895381572659312	1
837478	0.000876166160431524 1	10+
885605	0.0008216087428297469	6

可以发现 PageRank 算法确实有区别于直接按度数排序的方法。

2.3 Something fun

(声明：下面的时间数据是随手测的，没有非常严格地测量。没有特殊说明的话，测试数据集均为上面 Google 的数据集)

2.3 Something fun

(声明：下面的时间数据是随手测的，没有非常严格地测量。没有特殊说明的话，测试数据集均为上面 Google 的数据集)

我们手搓的稀疏矩阵乘法又 `Vec` 又 `HashMap`，还没上并行化，看起来速度应该是打不过经过工业级优化的 NumPy 的。

2.3 Something fun

(声明：下面的时间数据是随手测的，没有非常严格地测量。没有特殊说明的话，测试数据集均为上面 Google 的数据集)

我们手搓的稀疏矩阵乘法又 `Vec` 又 `HashMap`，还没上并行化，看起来速度应该是打不过经过工业级优化的 NumPy 的。

但是实际测试下：

- Rust 手搓：1.0029702s
- NetworkX（基于 scipy 和 numpy）：6.83s

2.3 Something fun

理论上 NumPy 是基于 C/C++ 的，不应该打不过 Rust。我们猜想，可能即使矩阵乘法那一块加速了，但是 Python 本身还是比较拉跨。

2.3 Something fun

理论上 NumPy 是基于 C/C++ 的，不应该打不过 Rust。我们猜想，可能即使矩阵乘法那一块加速了，但是 Python 本身还是比较拉跨。

比如 NetworkX 实现的 PageRank 算法在最后进行了一个数据转换：

```
return dict(zip(nodelist, map(float, x)))
```

2.3 Something fun

理论上 NumPy 是基于 C/C++ 的，不应该打不过 Rust。我们猜想，可能即使矩阵乘法那一块加速了，但是 Python 本身还是比较拉跨。

比如 NetworkX 实现的 PageRank 算法在最后进行了一个数据转换：

```
return dict(zip(nodelist, map(float, x)))
```

这个过程大概有 0.25s 的开销。

2.3 Something fun

同时我们在分析性能差距的时候偶然发现 NetworkX 有两处向量元素求和用的是 Python 自带的 `sum()`，而不是 NumPy 的 `sum()`。

2.3 Something fun

同时我们在分析性能差距的时候偶然发现 NetworkX 有两处向量元素求和用的是 Python 自带的 `sum()`，而不是 NumPy 的 `sum()`。

后者理论上应该会存在优化。

2.3 Something fun

```
start = time.perf_counter()
sum1 = np.sum(x[is_dangling])
sum_time1 = time.perf_counter() - start
start = time.perf_counter()
sum2 = sum(x[is_dangling])
sum_time2 = time.perf_counter() - start
start = time.perf_counter()
sum3 = sum(x[is_dangling])
sum_time3 = time.perf_counter() - start
start = time.perf_counter()
sum4 = np.sum(x[is_dangling])
sum_time4 = time.perf_counter() - start
print(f"Sum Time: {sum_time1:.10f} seconds, {sum_time2:.10f} seconds, {
    sum_time3:.10f} seconds, {sum_time4:.10f} seconds")
```

2.3 Something fun

```
Sum Time: 0.0007621000 seconds, 0.0040127000 seconds, 0.0037306000
     seconds, 0.0005124000 seconds
Sum Time: 0.0008018000 seconds, 0.0041156000 seconds, 0.0040253000
     seconds, 0.0007563000 seconds
```

2.3 Something fun

```
Sum Time: 0.0007621000 seconds, 0.0040127000 seconds, 0.0037306000
     seconds, 0.0005124000 seconds
Sum Time: 0.0008018000 seconds, 0.0041156000 seconds, 0.0040253000
     seconds, 0.0007563000 seconds
```

测试结果表明使用 NumPy 的 `sum()` 大概会有 3 到 4 倍的提升。

2.3 Something fun

因此我们顺手贡献了一下代码（现已合并到主分支）：

perf: replace built-in sum() with np.sum() in pagerank and katz #8561

Code

Merged dschult merged 2 commits into `networkx:main` from `caiwen666:perf/use_numpy_sum` last week

Conversation 5 | Commits 2 | Checks 42 | Files changed 2 | +5 -2

caiwen666 commented last week — Contributor

This pull request makes minor improvements to two centrality algorithms in the NetworkX library by replacing the built-in `sum` function with NumPy's `np.sum` for better performance and consistency with NumPy arrays.

Algorithm consistency and performance improvements:

- `networkx/algorithms/centrality/katz.py`: Updated the normalization step in the `katz_centrality_numpy` function to use `np.sum` instead of `sum` when calculating the sign of the centrality vector.
- `networkx/algorithms/link_analysis/pagerank_alg.py`: Modified the power iteration step in the `_pagerank_scipy` function to use `np.sum` instead of `sum` for aggregating dangling node values.

caiwen666 added 2 commits last week

`perf: replace built-in sum() with np.sum() in pagerank and katz` ✗ cf4de5c

`chore: fmt` ✓ ef1677b

Reviewers
rossbar ✓
amcandio ✓

Assignees
No one assigned

Labels
type: Enhancements

Milestone
3.7

Development
Successfully merging this pull request may close these issues.
None yet

2.3 Something fun

我们在腾讯云计算型 C4 实例上进行了非常粗糙的性能测试。

2.3 Something fun

我们在腾讯云计算型 C4 实例上进行了非常粗糙的性能测试。

方法和对比结果在
https://github.com/networkx/networkx/pull/8561

2.3 Something fun

我们在腾讯云计算型 C4 实例上进行了非常粗糙的性能测试。

方法和对比结果在
https://github.com/networkx/networkx/pull/8561

在 https://snap.stanford.edu/data/ego-Twitter.html 的数据集上，通过设置 `alpha=0.99`，即上文的 $followingProb$，`tol=1e-12`，即上文的 $tolerance$，人为地把迭代次数拉高到了 1135。

2.3 Something fun

我们在腾讯云计算型 C4 实例上进行了非常粗糙的性能测试。

方法和对比结果在
https://github.com/networkx/networkx/pull/8561

在 https://snap.stanford.edu/data/ego-Twitter.html 的数据集上，通过设置 `alpha=0.99`，即上文的 $followingProb$，`tol=1e-12`，即上文的 $tolerance$，人为地把迭代次数拉高到了 1135。

获得了大概 10% 的性能提升。

3. 应用

PageRank 本质上还是图上权重的传播。通过将要分析的东西构造成图，以及设置 $\mathbf{V}_0$、**danglingWeights**、**personalization** 等参数，可以发挥出不同的作用。

3.1 TrustRank

TrustRank 用于检测垃圾网站。

3.1 TrustRank

TrustRank 用于检测垃圾网站。

TrustRank 基于这样一个观察：权威的网站一般不会指向垃圾网站。所以 TrustRank 算法先挑选一些完全可以信任的网站，将这些网站的 TrustRank 的值设为最高，其他网站的 TrustRank 通过这些网站的 TrustRank 值传播得到。

3.1 TrustRank

设选择了 n 个完全信任的网站，那么：

$$\mathbf{V}_0(i) = \mathbf{personalization}(i) = \begin{cases} \frac{1}{n} & \text{, i 为完全信任的网站} \\ 0 & \text{, otherwise} \end{cases}$$

3.1 TrustRank

Anti-TrustRank：反过来做，标记垃圾网站，然后从垃圾网站逆向传播权重。

3.2 TextRank

TextRank 算法用于给文本内的词计算排名，可以统计文本的关键字，生成摘要。

3.2 TextRank

TextRank 算法用于给文本内的词计算排名，可以统计文本的关键字，生成摘要。

首先先把文本进行分词，比如：

3.2 TextRank

TextRank 算法用于给文本内的词计算排名，可以统计文本的关键字，生成摘要。

首先先把文本进行分词，比如：

党中央决定，在全党开展树立和践行正确政绩观学习教育，这是今年党的建设的重要任务

3.2 TextRank

TextRank 算法用于给文本内的词计算排名，可以统计文本的关键字，生成摘要。

首先先把文本进行分词，比如：

```
党中央决定，在全党开展树立和践行正确政绩观学习教育，这是今年党的建设的重要任务

['党中央', '决定', ', ', '在', '全党', '开展', '树立', '和', '践行', '正确', '政绩观', '学习', '教育', ', ', '这是', '今年', '党的建设', '的', '重要', '任务']
```

3.2 TextRank

分词会分出来一些标点符号和无关紧要的词（比如“的”、“和”这种）。我们的实现是直接把长度为 1 的词全部过滤掉：

3.2 TextRank

分词会分出来一些标点符号和无关紧要的词（比如“的”、“和”这种）。我们的实现是直接把长度为 1 的词全部过滤掉：

```
['党中央', '决定', '全党', '开展', '树立', '践行', '正确', '政绩观', '学习
    ', '教育', '这是', '今年', '党的建设', '重要', '任务']
```

3.2 TextRank

算法有一个参数 $windowSize$。我们在分词出来的序列上从左到右用大小为 $windowSize$ 的窗口扫描，然后把窗口内的词两两建立无向边。

3.2 TextRank

算法有一个参数 $windowSize$。我们在分词出来的序列上从左到右用大小为 $windowSize$ 的窗口扫描，然后把窗口内的词两两建立无向边。

比如 $windowSize = 2$ 时，会建立如下的边：

```
党中央 <-> 决定
决定 <-> 全党
全党 <-> 开展
...
```

3.2 TextRank

算法有一个参数 $windowSize$。我们在分词出来的序列上从左到右用大小为 $windowSize$ 的窗口扫描，然后把窗口内的词两两建立无向边。

比如 $windowSize = 2$ 时，会建立如下的边：

```
党中央 <-> 决定
决定 <-> 全党
全党 <-> 开展
...
```

然后就可以得到一张图，对这个图跑 PageRank 就得到了 TextRank。

3.2 TextRank

我们测试了一下效果，在 http://dangjian.people.com.cn/n1/2026/0225/c117092-40669694.html 这篇文章上做 $windowSize = 3$ 的 TextRank，得到结果：

3.2 TextRank

我们测试了一下效果，在 http://dangjian.people.com.cn/n1/2026/0225/c117092-40669694.html 这篇文章上做 $windowSize = 3$ 的 TextRank，得到结果：

词	TextRank	词频排名 (词频)
教育	0.01689736222299958	2(14)
学习	0.01596259207243153 4	2(14)
坚持	0.01435974021719568	7(8)
人民	0.01364675522987989 8	7(8)
政绩观	0.01345246004526632 8	1(15)

3.2 TextRank

然后又在某个小作文上面跑了一遍 $windowSize = 3$ 的 TextRank

3.2 TextRank

得到结果:

词	TextRank	词频排名 (词频)
我们	0.024564948289610245	1(44)
没有	0.008737309221451349	4(14)
放假	0.007786354788189854	2(15)
什么	0.007207502319517551 5	2(15)

3.2 TextRank

TextRank 还可以通过搞什么余弦相似度来把句子之间建图，生成文章摘要。不过我们不太感兴趣了。现在我们随便调用一下大模型就能得到更优的结果。

3.2 TextRank

TextRank 还可以通过搞什么余弦相似度来把句子之间建图，生成文章摘要。不过我们不太感兴趣了。现在我们随便调用一下大模型就能得到更优的结果。

比如上面的关键字丢给大模型：

关键字（均可在原文中找到）

- 初恋
- 同桌
- 十字路口
- 表白

3.2 TextRank

TextRank 的代码实现在代码仓库的 `src/bin/text_rank.rs` 里，感兴趣的可以玩玩。

THANK YOU!